Plantillas del reto

Aquí tenéis todas las plantillas del reto. Copiad cada bloque en un fichero con el nombre que indica el título, en el repositorio de vuestra app, y adaptadlo a vuestro problema. Las dos rutas de despliegue valen igual, así que usad solo la carpeta de la ruta que hayáis elegido.

📝 Lo que escribís

Ficha del hito de la semana 2 (se publica en el debate)

```
# Ficha del hito de la semana 2

> Copiad estas cinco líneas, rellenadlas y publicadlas como respuesta en el debate **Hito de la semana 2** de Blackboard. No tiene nota: sirve para que os devuelva feedback en la sesión 3 y para que vuestros compañeros encuentren vuestra app y la prueben en la semana 3.

1. **Problema:** [una frase]
2. **Usuario:** [quién lo tiene, en una frase]
3. **Qué hará la app:** [hasta tres funcionalidades]
4. **Pieza de IA:** [API de Gemini · modelo de Hugging Face · RAG] porque [una frase]
5. **Enlace a la versión 0:** [https://…streamlit.app o https://…hf.space]
```

especificacion.md

```
# Especificación · [Nombre del producto]

> Una buena especificación es la que otro asistente de IA podría construir sin haceros preguntas. Rellenadla en la semana 2 y usadla como prompt principal con vuestro asistente o agente. De ella sale la ficha del **hito de la semana 2**.

## 1. Problema y usuario

- **Quién tiene el problema:** [persona o colectivo concreto]
- **Cómo lo resuelve hoy:** [a mano, con Excel, preguntando a alguien…]
- **Qué le molesta de cómo lo hace hoy:** [tiempo, errores, no encuentra la información…]
- **Qué parte resuelve la IA:** [y qué parte se queda fuera]
```

2. Qué hace el producto

Máximo tres funcionalidades. Si hay más, van al apartado 8.

1. [Funcionalidad 1]
2. [Funcionalidad 2]
3. [Funcionalidad 3]

3. Entradas y salidas

El usuario da…	La app devuelve…	Ejemplo
[un texto, una pregunta, un archivo CSV…]	[un resumen, una respuesta, una categoría…]	[ejemplo concreto]

4. La pieza de IA

- **Opción elegida:** [API de un LLM (Gemini) · modelo de Hugging Face · RAG sobre nuestros documentos]
- **Por qué:** [qué gana la app con esa opción]
- **Instrucciones para el modelo (system prompt):** [cómo debe comportarse, tono, qué no debe hacer]

5. Flujo del usuario

1. [El usuario entra y ve…]
2. [Escribe o sube…]
3. [Recibe…]

6. Restricciones

- Coste cero: capa gratuita de la API y despliegue gratuito (ruta A o B).
- La clave de la API va en secretos, nunca en el código.
- Sin datos personales reales.
- Lenguaje sencillo, para alguien sin conocimientos técnicos.
- [Otras restricciones de vuestro problema]

7. Criterios de aceptación

La app está bien si:

| Entrada | Resultado esperado |

```
|---|---|
| [caso normal] | [qué debe pasar] |
| [caso límite: texto vacío, pregunta fuera de tema…] | [qué debe pasar] |
| [error: se acaba la cuota] | [mensaje claro para el usuario] |

## 8. Fuera de alcance (por ahora)

- [Ideas que dejáis para después]

## 9. Despliegue

- **Ruta:** [A (Streamlit Community Cloud) · B (Hugging Face Spaces)]
- **Si trabajáis en equipo (opcional):** [qué app o funcionalidad construye cada persona]
```

AGENTS.md

```
# AGENTS.md · [Nombre de la app]

> Este fichero lo leen los agentes de programación (Copilot, Cursor, Codex, Gemini CLI y otros) antes de tocar vuestro código. Para Claude Code, cread un `CLAUDE.md` con una sola línea: `Sigue las instrucciones de AGENTS.md`. Para Gemini CLI, haced lo mismo con `GEMINI.md`.

## Qué es este proyecto

[Una frase: qué problema resuelve y para quién. Ejemplo: "Asistente que responde las dudas de los vecinos de la asociación X a partir de sus estatutos".]

## Cómo está hecho

- Python 3.12.
- Interfaz: [Streamlit (ruta A) | Gradio (ruta B)].
- IA: API de Gemini con el SDK `google-genai`. El modelo está en la constante `MODELO` de `app.py`.
- Archivos: `app.py` (toda la app), `requirements.txt` (dependencias) y `README.md`.
- La especificación completa está en [enlace a vuestra especificación].

## Reglas

- La clave de la API va SIEMPRE en secretos (`st.secrets` en la ruta A,
```

```
variable de entorno `GEMINI_API_KEY` en la ruta B). Nunca en el código, en el README ni en el chat.
- Cada llamada a la API va dentro de `try/except` y muestra un mensaje claro para el usuario.
- Textos de la interfaz en español, sencillos, pensados para alguien sin conocimientos técnicos.
- Cambios pequeños: uno por petición. Al terminar, explica qué has cambiado y por qué.
- Si añades una librería, añádela también a `requirements.txt` y dilo.
- No uses datos personales reales. Para probar, inventa ejemplos.

## Cómo comprobar que funciona

- Ruta A: `streamlit run app.py` en local (con `.streamlit/secrets.toml`), o haz push y revisa la app en Streamlit Cloud.
- Ruta B: `python app.py` con `GEMINI_API_KEY` exportada, o sube los archivos al Space.
- Casos de prueba (entrada → lo que esperamos):
  1. [entrada] → [respuesta esperada]
  2. [entrada] → [respuesta esperada]
  3. [entrada fuera de tema] → [la app dice amablemente que no puede ayudar con eso]

## Qué no hacer

- No borres ni reescribas el README sin preguntar.
- No cambies el modelo ni la arquitectura sin preguntar.
- No hagas push ni despliegues sin que yo lo haya revisado.
```

README.md de vuestra app

```
# [Nombre de la app]

**Enlace a la app:** [https://…streamlit.app o https://…hf.space]
**Autor/a:** [vuestro nombre]

## Pitch

[Cinco frases: el problema y quién lo tiene, qué hace la app, qué aporta la IA, qué aprendisteis de la prueba y qué haríais después, con un dato o un ejemplo.]

## El problema
```

````
[Quién tiene el problema, cómo lo resuelve hoy y por qué merece la pena resolverlo. 3 o 4 frases.]

## Qué hace la app

[Qué puede hacer el usuario, paso a paso.]

## La pieza de IA

- **Qué usa:** [API de Gemini con el modelo …, un modelo de Hugging Face …, un RAG sobre …]
- **Por qué esa opción:** [una o dos frases]
- **Qué pasa si falla:** [cómo responde la app si se acaba la cuota o no hay conexión]

## Cómo la he construido

- **Herramientas:** [Gemini en Colab, Copilot, Cursor, Claude Code, Codex, Gemini CLI…]
- **Fichero de contexto del agente:** [enlace a vuestro AGENTS.md, CLAUDE.md o GEMINI.md]
- **Prompts clave:** [los 3 a 5 prompts que más han servido, copiados tal cual]
- **Qué delegué al agente y qué revisé a mano:** [ejemplos concretos]

## Prueba y mejora

| Quién la probó | Qué probó | Qué pasó | Qué cambié |
|---|---|---|---|
| [un compañero, o yo como usuario] | | | |

## Ejecutar en local (opcional)

```bash
pip install -r requirements.txt
# Ruta A: poned la clave en .streamlit/secrets.toml y ejecutad
streamlit run app.py
# Ruta B: exportad la clave y ejecutad
export GEMINI_API_KEY="…" && python app.py
```

## Capturas
````

```
[La app funcionando, la app ante un caso límite, un momento de vuestro trabajo con la IA y la prueba. Sin ninguna clave a la vista.]
```

Documento de la entrega final (ocho apartados)

```
# Entrega final · [Nombre de la app]

> Copiad esta plantilla en un documento, rellenad los ocho apartados en este orden y entregadlo en `.docx` o `.pdf` en la Actividad Sesión 4 de Blackboard. Cada apartado corresponde a un criterio de la rúbrica. Casi todo lo podéis copiar del README de vuestra app.

**Nombre y apellidos:** [vuestro nombre]

## 1. Enlaces

- **App desplegada:** [https://…streamlit.app o https://…hf.space]
- **Repositorio:** [https://github.com/…]

## 2. Problema y usuario

[Quién tiene el problema, cómo lo resuelve hoy sin la app, y qué parte resuelve la IA y qué parte queda fuera. 3 o 4 frases.]

## 3. Especificación

[Pegad vuestra especificación o poned el enlace al fichero en el repositorio. Tiene que incluir las funcionalidades, las entradas y salidas, y los criterios de aceptación con algún caso límite.]

## 4. La pieza de IA

- **Qué usa:** [API de Gemini con el modelo …, un modelo de Hugging Face …, un RAG sobre …]
- **Por qué esa opción:** [una o dos frases]
- **Qué pasa si falla:** [cómo responde la app si se acaba la cuota o no hay conexión]
- **Dónde está la clave:** [en los secretos de la plataforma, nunca en el código]

## 5. Proceso con IA
```

- **Herramientas:** [Gemini en Colab, Copilot, Cursor, Claude Code, Codex, Gemini CLI…]
- **Fichero de contexto del agente:** [enlace a vuestro AGENTS.md, CLAUDE.md o GEMINI.md]
- **Prompts clave:** [los 3 a 5 prompts que más han servido, copiados tal cual]
- **Qué delegué al agente y qué revisé a mano:** [ejemplos concretos]

6. Prueba y mejora

Quién la probó	Qué probó	Qué pasó	Qué cambié
[un compañero, o yo como usuario]			

- **Feedback que he dado a un compañero** (opcional): [a quién, y copiad lo que le escribisteis en el debate del hito]

7. Pitch escrito

1. [El problema y quién lo tiene.]
2. [Qué hace la app.]
3. [Qué aporta la IA, que no se conseguiría sin ella.]
4. [Qué aprendí de la prueba y qué cambié.]
5. [Qué haría después, con un dato o un ejemplo concreto que lo respalde.]

8. Capturas

Cuatro capturas de pantalla, cada una con una línea que diga qué se ve. Me gusta mucho verlas, porque enseñan cómo habéis trabajado mejor que cualquier explicación. Que no se vea ninguna clave en ninguna.

1. **La app funcionando** en su caso principal.
   [captura]
2. **La app ante un caso límite o un error:** un texto vacío, una pregunta fuera de tema o la cuota agotada, con su mensaje.
   [captura]
3. **Vuestro trabajo con la IA:** un momento de la conversación con el asistente o el agente, donde se vea uno de vuestros prompts clave y lo que os devolvió.
   [captura]
4. **La prueba:** la respuesta de un compañero en el debate del hito o, si la habéis probado vosotros, la app con el caso que os hizo cambiar algo.
   [captura]

Si queréis añadir alguna más, del antes y el después de un cambio por ejemplo, adelante.

🅰️ Ruta A · Streamlit Community Cloud

app.py

```
"""Plantilla de app del reto · Ruta A (Streamlit Community Cloud).

Chat con Gemini que recuerda la conversación. Para adaptarla a vuestro problema:
1. Cambiad TITULO, DESCRIPCION y SYSTEM_PROMPT.
2. Si vuestra app no es un chat (por ejemplo, un formulario que resume un texto),
   pedidle a vuestro asistente de IA que la transforme a partir de vuestra especificación.
"""

import streamlit as st
from google import genai
from google.genai import types

MODELO = "gemini-3.8-flash"  # si da error 429 (cuota agotada), probad "gemini-3.5-flash-lite"
TITULO = "🤖 Mi app del reto"
DESCRIPCION = "Explicad aquí en una frase qué hace la app y para quién es."
SYSTEM_PROMPT = (
    "Eres un asistente amable que ayuda a [QUIÉN] con [QUÉ PROBLEMA]. "
    "Responde en español, con frases cortas y claras. Si no sabes algo, dilo."
)

st.set_page_config(page_title=TITULO, page_icon="🤖")
st.title(TITULO)
st.caption(DESCRIPCION)

# La clave NUNCA va en el código: se guarda en Settings → Secrets de Streamlit Cloud
try:
    clave = st.secrets["GEMINI_API_KEY"]
except Exception:
    st.error("Falta el secreto GEMINI_API_KEY. Añadidlo en Settings →
```

```
Secrets de vuestra app.")
    st.stop()


@st.cache_resource
def obtener_cliente(api_key: str) -> genai.Client:
    """Crea el cliente de Gemini una sola vez y lo reutiliza entre
peticiones."""
    return genai.Client(api_key=api_key)


if "mensajes" not in st.session_state:
    st.session_state.mensajes = []

for mensaje in st.session_state.mensajes:
    with st.chat_message(mensaje["role"]):
        st.markdown(mensaje["content"])

if pregunta := st.chat_input("Escribe tu pregunta"):
    st.session_state.mensajes.append({"role": "user", "content": pregunta})
    with st.chat_message("user"):
        st.markdown(pregunta)

    # Gemini llama "model" a lo que Streamlit llama "assistant"
    contenidos = [
        types.Content(
            role="user" if m["role"] == "user" else "model",
            parts=[types.Part(text=m["content"])],
        )
        for m in st.session_state.mensajes
    ]
    try:
        respuesta = obtener_cliente(clave).models.generate_content(
            model=MODELO,
            contents=contenidos,

config=types.GenerateContentConfig(system_instruction=SYSTEM_PROMPT),
        )
        texto = respuesta.text or "No he recibido respuesta. Probad otra
vez."
    except Exception as error:  # cuota agotada, clave incorrecta, red
caída...
        texto = f"⚠️ Ahora mismo no puedo responder ({error}). Probad de
```

```
nuevo en un minuto."

    with st.chat_message("assistant"):
        st.markdown(texto)
    st.session_state.mensajes.append({"role": "assistant", "content":
texto})
```

requirements.txt

```
streamlit
google-genai
```

.streamlit/secrets.toml (solo en local, nunca en GitHub)

```
# Para probar en local: copiad este archivo a .streamlit/secrets.toml y
poned vuestra clave.
# .streamlit/secrets.toml está en .gitignore, así que no se sube a GitHub.
# En Streamlit Cloud, pegad esta misma línea en Settings → Secrets.
GEMINI_API_KEY = "pegad-aqui-vuestra-clave"
```

🅱️ Ruta B · Hugging Face Spaces con Gradio

README.md (cabecera del Space)

```
---
title: Mi App del Reto
emoji: 🤖
colorFrom: blue
colorTo: green
sdk: gradio
sdk_version: 6.27.0
python_version: "3.12"
app_file: app.py
pinned: false
---

# Mi app del reto

> Hugging Face lee la cabecera de arriba (entre las líneas `---`) para saber
cómo arrancar la app. No la borréis; cambiad solo `title` y `emoji`.

El resto de este README sigue la plantilla [`README_app.md`]
(../README_app.md): copiadla aquí debajo y rellenadla.
```

app.py

```
"""Plantilla de app del reto · Ruta B (Hugging Face Spaces con Gradio).

Chat con Gemini que recuerda la conversación. Para adaptarla a vuestro
problema:
1. Cambiad TITULO, DESCRIPCION, EJEMPLOS y SYSTEM_PROMPT.
2. Si vuestra app no es un chat, pedidle a vuestro asistente de IA que la
transforme
   (por ejemplo, a un gr.Interface con un cuadro de texto de entrada y otro
de salida).
"""

import os

import gradio as gr
from google import genai
from google.genai import types

MODELO = "gemini-3.8-flash"  # si da error 429 (cuota agotada), probad
"gemini-3.5-flash-lite"
TITULO = "🤖 Mi app del reto"
DESCRIPCION = "Explicad aquí en una frase qué hace la app y para quién es."
EJEMPLOS = ["¿Qué puedes hacer por mí?", "Ponme un ejemplo de uso"]
SYSTEM_PROMPT = (
    "Eres un asistente amable que ayuda a [QUIÉN] con [QUÉ PROBLEMA]. "
    "Responde en español, con frases cortas y claras. Si no sabes algo,
dilo."
)

_cliente: genai.Client | None = None


def obtener_cliente() -> genai.Client:
    """Crea el cliente la primera vez. La clave llega como secreto del
Space."""
    global _cliente
    if _cliente is None:
        clave = os.environ.get("GEMINI_API_KEY")
        if not clave:
            raise RuntimeError("falta el secreto GEMINI_API_KEY en Settings
→ Variables and secrets")
        _cliente = genai.Client(api_key=clave)
```

```
    return _cliente


def _texto(contenido) -> str:
    """Gradio 6 puede entregar el contenido como texto o como lista de
bloques."""
    if isinstance(contenido, str):
        return contenido
    if isinstance(contenido, list):
        return " ".join(b.get("text", "") for b in contenido if
isinstance(b, dict))
    return str(contenido)


def responder(mensaje: str, historial: list[dict]) -> str:
    """Envía a Gemini toda la conversación y devuelve su respuesta."""
    contenidos = [
        types.Content(
            role="user" if h["role"] == "user" else "model",
            parts=[types.Part(text=_texto(h["content"]))],
        )
        for h in historial
    ]
    contenidos.append(types.Content(role="user", parts=
[types.Part(text=mensaje)]))
    try:
        respuesta = obtener_cliente().models.generate_content(
            model=MODELO,
            contents=contenidos,

config=types.GenerateContentConfig(system_instruction=SYSTEM_PROMPT),
        )
        return respuesta.text or "No he recibido respuesta. Probad otra
vez."
    except Exception as error:  # cuota agotada, clave incorrecta, red
caída...
        return f"⚠️ Ahora mismo no puedo responder ({error}). Probad de
nuevo en un minuto."


demo = gr.ChatInterface(fn=responder, title=TITULO, description=DESCRIPCION,
examples=EJEMPLOS)
```

```
if __name__ == "__main__":
    demo.launch()
```

requirements.txt

```
google-genai
```